EXPANDED REFERENCE / VERSION 3.1.2

Command Language Guide

SpinDoctor 3.1.2
Vocabulary, dictionary, grammar, and practical workflows

Reference edition for the **298-qubit update**
Update identity: `3.1.2-298-20261010`
Prepared October 10, 2026 (Pacific Time).

Learn the language. Choose the right calculation.

This guide expands the supplied Hand Guide into a working language reference. It explains which words are commands, how their arguments are parsed, what each command acts on, and how to turn a program into a completed result.

Inside this edition	What it gives you
Command dictionary	Every validated calculator action, operational terminal families, and the separate virtual dashboard vocabulary.
Gate dictionary	33 canonical coherent-register gates, operand counts, aliases, and control order.
Grammar	Positional commands, key=value fields, expressions, lists, matrices, and file schemas.
Examples	Bell, rotation, GHZ, wide registers, CSV fitting, and ordered folder workflows.
Scope and limits	Logical width, joint factors, retained output, job completion, and verification scope.

Authority and use

The updated local source and release notes take precedence over older prose. The supplied PDF still describes the original 29-qubit build in places; its final-page update note does not revise those earlier tables. This edition reconciles those differences.

Use fresh circuit names. Each original command-block line is a separate submission unless labelled as a file. Long lines wrap visually in this PDF; HTML copy buttons preserve their original form. Templates contain placeholders; examples contain values. This guide documents the application without changing its parser or your saved session.

NAVIGATION

Find what you need

Read the first six chapters to learn the language. Use the dictionaries when you already know the calculation you want. The PDF has bookmarks; the HTML companion adds chapter search and copy buttons.

Chapter	p.	Chapter	p.
01 / Start a session and run a circuit	3	19 / Compilation and algorithm dictionary	32
02 / Choose a command family	4	20 / Memory controls and analysis limits	34
03 / Words, fields, and numeric values	5	21 / Attachment command dictionary	35
04 / Grammar and typed argument dictionary	6	22 / Program-file grammar	36
05 / Expression rules and bit order	7	23 / QASM and PDF input boundaries	37
06 / Gate dictionary: local gates	8	24 / CSV dictionary and schemas	38
07 / Gate dictionary: joint gates	9	25 / Presets, sequences, and folder runs	39
08 / Circuit command dictionary	10	26 / Ordered jobs and repeatable file packs	40
09 / Built-in presets and sampling modes	11	27 / Worked language recipes	41
10 / Tranche command dictionary	12	28 / Grammar mistakes and their corrections	42
11 / Procedural command dictionary	13	29 / Reports, results, and provenance	43
12 / Query the completed circuit	14	30 / Separate virtual dashboard language	44
13 / State and matrix calculator dictionary	17	31 / Launcher and package-tool vocabulary	45
14 / Resource planning dictionary	22	32 / Troubleshooting dictionary	46
15 / Physics and model dictionary	24	33 / Vocabulary dictionary: A-M	47
16 / Lesson dictionary	26	34 / Vocabulary dictionary: N-Z	49
17 / Noise and mitigation dictionary	28	35 / Source map, version notes, and validation	51
18 / Error-correction dictionary	30	36 / Validation evidence and practical scope	52

Search examples: `mcx7`, `q.math`, `collapse`, `target-unit`, `purge`, `prepared-factor-Born-Z`. Uppercase words in templates are values to replace. Square brackets in syntax templates mark optional parts; in an actual command, square brackets construct a list or matrix.

REFERENCE 02

01 / Start a session and run a circuit

Launch the updated engine

Open the release folder, open its `SpinDoctor` subfolder, and launch `BOOT.cmd`. The launcher verifies the TIFF carrier, refreshes its program projection, and opens the terminal. `BOOT.cmd gif` selects the GIF carrier. Keep the launcher console open.

To activate updated code, stop an older launcher with Ctrl+C and launch `BOOT.cmd` again. A browser refresh cannot replace an already compiled engine. Keep the session lock file; the handle, rather than the presence of the file, controls access.

```
q.status
q.help
```

In `q.status`, check `coherentRegisterCapabilities`: updateId `3.1.2-298-20261010`, maximumLogicalQubits `298`, maximumJointFactorQubits `59`, and globalIndexEncoding `BigInteger`.

Make a Bell pair

```
q.circuit new guide_bell 2
q.circuit add h 0
q.circuit add cx 0 1
q.circuit run shots 1024 basis z
```

The first line creates and selects a two-qubit program. H puts qubit 0 into a superposition; CX uses qubit 0 as the control and qubit 1 as the target. The run queues simulation and sampling.

```
q.circuit status
q.circuit result
```

Repeat the status check until the job is inactive and its phase reports completion. A queued response means accepted work, not a finished histogram. A completed ideal Bell circuit has only `00` and `11` outcomes, with expected probabilities 1/2 each; finite shot counts vary.

Inspect the completed state

```
q.query probabilities max=4
q.query expectation pauli="Z0 Z1"
q.circuit spin 0
q.history list
```

The ideal ZZ expectation is +1. Each individual Bell qubit has a zero Bloch vector because its reduced state is mixed. Download the saved TXT, PDF, or CSV report from the report controls.

If guide_bell already exists, choose a fresh name or select it. Adding new gates or changing input invalidates its retained output; run it again before a state query.

REFERENCE 03

02 / Choose a command family

You want to...	Use	The command acts on
Build and sample a gate program	`q.circuit`	The selected named circuit.
Inspect the last coherent circuit output	`q.query`	The selected circuit, after its completed run.
Manipulate a live register directly	`q.tranche`	A separate selected register and replay journal.
Describe many independent indexed spinors	`q.prepare`, `q.migrate`, `q.gate`	The procedural grid descriptor.
Calculate from explicit vectors/matrices	`q.math`	Values supplied in the command.
Estimate memory or sampling error	`q.resource`	Declared assumptions; estimates do not allocate a state.
Explore Hamiltonians or model lessons	`q.model`, `q.lesson`	Small calculations or educational explanations.
Calculate noise, correction, or compilation	`q.noise`, `q.qec`, `q.compile`	Explicit small states/programs and stated conventions.
Fit a measurement table or import a file	`q.input`	An explicitly uploaded attachment ID.
Run ordered commands or a folder	`q.preset`, `q.sequence`, `q.folder`	A background sequence with child jobs and reports.
Inspect saved command records	`q.history`	Archived command reports.
Use the virtual domain ledger	VXPU dashboard tokens	A separate browser virtual epoch engine.

Three meanings of state

A procedural address describes one independent qubit from a formula. A circuit describes an ordered program and retains a calculated coherent output after a run. A tranche is a live register that its gate and collapsing-measurement commands can change directly. These have separate controls.

`q.migrate t 1` changes the procedural family and resets its composed matrix. It does not enlarge, select, or execute a circuit. `q.tranche from I` snapshots the I-th block of up to 29 procedural addresses; later tranche operations are independent of that family.

Read the right completion record

Use `q.circuit status`, `q.tranche status`, `q.verify status`, or `q.sequence status` for the job you started. Folder runs also expose `q.folder status`. `q.status` is the complete session snapshot. A status request observes progress; it does not wait for completion.

REFERENCE 04

03 / Words, fields, and numeric values

Submit one terminal command per line. Commands start with `q.` and use spaces between positional words. Write family names and actions in lowercase for consistency; many dispatchers accept other case, but circuit names and textual values have their own rules.

Grammar form	Example	How to read it
Positional	`q.circuit new demo 2`	Order is fixed: action, name, qubit count.
Positional labels	`q.circuit run shots 256 basis z`	shots and basis are literal words; values follow them.
Named fields	`q.math bloch amplitudes=[1,0]`	Use key=value; required and optional keys depend on the action.
Quoted text	`pauli="Z0 Z1"`	Quotes retain spaces inside one field.
List	`qubits=[0,1]`	Commas separate typed entries.
Matrix	`rho=[[1,0],[0,0]]`	Nested bracketed rows; dimensions are checked.
Gate script	`gates="h 0;cx 0 1"`	Semicolons separate gates inside a quoted field only.

The important numeric split

Context	Use	Example
Direct q.gate / q.circuit add / q.tranche gate	Finite decimal or scientific literals for angles	`q.circuit add ry 1.5707963267948966 0`
Calculator key=value expressions	Restricted arithmetic and functions	`q.math gate-matrix gate=ry angles=[pi/2]`
Text/CSV/QASM circuit source	Supported parameter expressions compiled by the input parser	`ry pi/2 0` in a text circuit.
Ordered sequences	Legacy gate-angle expressions can be normalized before dispatch	Prefer decimal direct-gate examples so they also work interactively.
Measurement CSV selected cells	Finite decimal/scientific numbers	`0.75` is numeric; `sqrt(3)/2` is excluded as nonnumeric.

Use a decimal point, no thousands separators, and explicit multiplication. `i/2` and `exp(i*pi/4)` are calculator expressions. `2i` is not a supported implicit product; write `2*i`. Direct positional indices/counts generally require decimal integers; some calculator integer fields accept integral expressions. Decimal integers work consistently across these contexts.

Avoid spaces around =, smart quotes, and copied Markdown fences in executable files. Double quotes are the safest quoting convention across terminal fields and sequence validation. Command files permit comments beginning with # or //; a comment is not an interactive terminal command.

REFERENCE 05

04 / Grammar and typed argument dictionary

The following is a compact grammar for the documented forms. It shows structure rather than every command-specific range. Quoted strings and nested lists are retained by the tokenizer; each family then validates its own arguments.

```
terminal     := family [SP action (SP argument)*]
field        := key "=" value
value        := atom | quoted-text | list | matrix
list         := "[" value ("," value)* "]"
matrix       := "[" list ("," list)* "]"
gate-line    := gate (SP angle)* (SP qubit)+
local-all    := local-gate (SP angle)* SP "all"
script       := gate-line (";" gate-line)*
expression   := sum
sum          := product (("+" | "-") product)*
product      := unary (("*" | "/") unary)*
unary        := "+" unary | "-" unary | power
power        := atom ["^" unary]
atom         := number | constant | "(" expression ")"
             | function "(" expression ")"
```

SP means whitespace. The script production applies inside quoted calculator fields or file formats, not at the top terminal level. Exact expression precedence is explained in the next chapter.

Placeholder or field	Type and meaning
NAME	Circuit identifier: ASCII letter first, then letters/digits/underscore/hyphen; 1..32 characters. Lookup is case-insensitive.
N, Q, INDEX	Whole numbers. N is a size; Q is a zero-based qubit; a basis INDEX is a nonnegative decimal integer below 2^N.
ANGLE	Direct gate: decimal radians. Calculator/input expression: safe real expression where allowed.
ID	The actual opaque attachment, report, preset, or job identifier returned by the application.
amplitudes, state, initial, eigenstate	Bracketed complex vector. Most operations require it already normalized; normalize explicitly.
rho, sigma, matrix, operator, Q, J	Bracketed matrix, or a permitted named zero-parameter gate matrix such as X, H, or I2.
qubits, partition, targets	Distinct zero-based integer list; selected query order is meaningful.
pauli, generators, stabilizers	Pauli text such as "X0 Z1"; collections use a bracketed list of quoted strings.
basis	x, y, or z where supported; circuit runs wider than 29 require z.
Boolean fields	true or false; some restricted backend options require true.
VALUE in dictionary templates	Replace with the action-specific typed value; use the example and allowed field list.

REFERENCE 06

05 / Expression rules and bit order

Restricted calculator expressions

Allowed constants are `pi`, `e`, and `i`. Functions are `sqrt`, `sin`, `cos`, `tan`, `exp`, `log`, `abs`, `asin`, `acos`, and `atan`. Operators are +, -, *, /, and ^; parentheses group expressions. Names are case-insensitive. The parser does not evaluate arbitrary host-language code.

```
q.math normalize amplitudes=[1,i]
q.math probabilities amplitudes=[sqrt(3)/2,i/2]
q.math from-bloch theta=pi/2 phi=pi/2
q.math unitary matrix=[[1,0],[0,exp(i*pi/4)]] tolerance=1e-10
```

Powers associate to the right: `2^3^2` means `2^(3^2)` and gives 512. Powers bind more tightly than unary signs: `-2^2` gives -4, `(-2)^2` gives 4, and `2^-2` gives 0.25. Explicit parentheses make the intended meaning clear.

Calculator expressions are bounded to 2,048 characters, 1,024 counted operations, and depth 32. Nonfinite results, division by zero, and unsupported names are rejected. A real-valued field rejects an imaginary residual above 1e-12. Vector and matrix shapes are independently bounded.

File angle/parameter expressions use a separate real parser: up to 1,024 characters, 512 operations, and depth 32. It supports lowercase pi/e and declared case-sensitive parameter names, with the same power/unary precedence. It does not accept complex i. q.input evaluate overrides can be expressions; q.input import overrides must be finite number literals.

Qubit 0 is the rightmost bit

Basis index for 3 qubits	Displayed state	Qubits equal to 1
0	`|000>`	None
1	`|001>`	q0
2	`|010>`	q1
4	`|100>`	q2
5	`|101>`	q2 and q0
7	`|111>`	q2, q1, and q0

The basis index is `sum(bit(q) * 2^q)`, and the displayed order is `|q(N-1)...q0>`. A state vector lists amplitudes in ascending basis-index order. For two qubits, `[a,b,c,d]` means the amplitudes of `00,01,10,11`.

Controlled-gate operands are control-first: `cx 0 1` controls q0 and targets q1. `ccx 0 1 2` has two controls; `cswap 0 1 2` controls q0 and swaps q1/q2. `mcx7` takes seven controls followed by one target. All gate operands must be distinct and within the selected register.

Wide indices use BigInteger decimal text. Preserve them and 298-bit outcome strings as text in spreadsheets; conversion to floating-point numbers can lose digits.

REFERENCE 07

06 / Gate dictionary: local gates

Use these gates with `q.circuit add GATE ...` or `q.tranche gate GATE ...`. Angles precede operands. Every gate in this chapter takes one qubit operand Q and supports `all` for a coherent register.

Gate	Angles	Meaning and direct example
`i`	0	Identity. `q.circuit add i 0`
`x`	0	Pauli X; flips 0 and 1. `q.circuit add x 0`
`y`	0	Pauli Y; bit flip with relative phases. `q.circuit add y 0`
`z`	0	Pauli Z; changes the phase of |1>. `q.circuit add z 0`
`h`	0	Hadamard. `q.circuit add h 0`
`s`, `sdg`	0	Phase +pi/2 and its adjoint. `q.circuit add sdg 0`
`t`, `tdg`	0	Phase +pi/4 and its adjoint. `q.circuit add tdg 0`
`sx`, `sxdg`	0	Square root of X and its adjoint. `q.circuit add sx 0`
`rx`, `ry`, `rz`	1	Rotation around the named axis. `q.circuit add ry 0.5 0`
`p`	1	Phase diag(1, exp(i*angle)). `q.circuit add p 0.5 0`
`u`	3	U(theta,phi,lambda). `q.circuit add u 0.5 0.2 0.1 0`

Canonical names and aliases

Alias	Canonical form	Where it belongs
`id`, `identity`	`i`	Rich circuit/tranche/calculator gate parsers.
`u3`	`u`	Same three angle operands.
`cnot`	`cx`	Control, target.
`toffoli`	`ccx`	Control, control, target.
`fredkin`	`cswap`	Control, swap operand, swap operand.

The procedural family has a smaller gate language

Direct `q.gate` accepts h, x, y, z, s, t, i, identity, ry, rz, u, and u3, always with `all`. It has no rx, p, adjoint-phase, swap, or controlled-gate forms. Do not transfer the entire coherent-register dictionary into this family.

```
q.gate ry 0.5 all
q.circuit add ry 0.5 all
q.tranche gate ry 0.5 all
```

These three lines have different state scopes. `p(angle)` and `rz(angle)` differ by a global phase on one isolated qubit; controlled versions can expose that phase difference. Use the named gate matching your intended convention.

REFERENCE 08

07 / Gate dictionary: joint gates

Gate	Angles / qubits	Operand order and effect
`cx`, `cy`, `cz`, `ch`	0 / 2	Control C, target T; apply X/Y/Z/H to T when C=1.
`swap`	0 / 2	A, B; exchange two qubits.
`cp`	1 / 2	Angle, C, T; controlled phase P.
`crx`, `cry`, `crz`	1 / 2	Angle, C, T; controlled axis rotation.
`rxx`, `ryy`, `rzz`, `rzx`	1 / 2	Angle, A, B; exp(-i*angle*PauliProduct/2). RZX uses Z operand first, X operand second.
`cu`	4 / 2	theta, phi, lambda, gamma, C, T; controlled exp(i*gamma)*U(theta,phi,lambda).
`ccx`	0 / 3	C0, C1, T; X if both controls are 1.
`cswap`	0 / 3	C, A, B; swap A/B when control is 1.
`mcx7`	0 / 8	C0 through C6, then T; X when all seven controls are 1.

```
q.circuit add cx 0 1
q.circuit add cp 0.7853981633974483 0 1
q.circuit add rzz 0.2 0 1
q.circuit add cu 0.5 0.2 0.1 0.0 0 1
q.circuit add ccx 0 1 2
q.circuit add cswap 0 1 2
q.circuit add mcx7 0 1 2 3 4 5 6 7
```

These are independent syntax examples; the selected circuit must be wide enough for each. Controlled and joint gates require explicit operands. `all` is accepted only for local one-qubit gates.

A connecting gate can grow the calculated joint factor

Registers contain independent coherent factors. A gate joining qubits from different factors can merge them. The updated engine supports at most 59 qubits in one joint factor and checks its storage against the memory budget before commit. A 298-qubit register does not imply an affordable arbitrary entangled state.

`q.math gate-matrix` can expose explicit matrices for supported gates up to four operands. `mcx7` runs through the native controlled-X implementation; an explicit 256x256 matrix is outside that calculator output limit.

Canonical total: 16 local gates, 14 two-qubit gates, two three-qubit gates, and mcx7 = 33 gates. Aliases do not add distinct operations.

REFERENCE 09

08 / Circuit command dictionary

Most circuit commands require a selected circuit. `new`, `select`, and built-in `preset` select one. Adding gates builds a definition; `run` performs the calculation.

Syntax	Effect and important rule
`q.circuit help`	Show native circuit help and supported gate metadata.
`q.circuit new NAME N`	Create and select a fresh program, N=1..298.
`q.circuit select NAME`	Select a saved circuit.
`q.circuit list`	List saved circuits and the selection.
`q.circuit show`	Show selected input basis and ordered gate list.
`q.circuit input INDEX`	Set starting computational basis index; 0 <= INDEX < 2^N.
`q.circuit add GATE ANGLES QUBITS`	Append one validated instruction; direct angles are numeric literals.
`q.circuit clear`	Remove selected gate list; the program remains.
`q.circuit delete [NAME]`	Delete named program, or the selected one if omitted.
`q.circuit capacity [N]`	Read/set program capacity; default 64, range 1..1024; cannot fall below saved count.
`q.circuit prune presets [KEEP]`	Remove oldest catalogue-preset-owned programs, retaining KEEP (default 0). User programs are retained.
`q.circuit memory MIB`	Set combined retained/working-state budget; at least 2 MiB and enough for retained output.
`q.circuit memory auto`	Choose a budget from measured headroom; no extra options in this command.
`q.circuit run shots N basis B`	Queue 1..100000 shots. x/y/z for up to 29 qubits; z for wider programs.
`q.circuit status`	Inspect selected circuit and current job.
`q.circuit result`	Inspect the last job/histogram; available=false when absent.
`q.circuit cancel`	Request cancellation; poll until work stops.
`q.circuit state INDEX`	One amplitude in the selected program's retained premeasurement output.
`q.circuit spin Q`	Single-qubit reduced Bloch vector of retained output.
`q.circuit export`	Export selected program, seed/counter metadata, and status.

State/spin/query require retained output matching the selected program hash. Histograms survive restart; the coherent output is recalculated by the next run. Circuit program storage is limited to 8192 instructions per program and 65536 across saved programs.

REFERENCE 10

09 / Built-in presets and sampling modes

```
q.circuit preset bell [2] [as NAME] [shots N] [basis B] [run=false]
q.circuit preset ghz [N] [as NAME] [shots N] [basis B] [run=false]
q.circuit preset qft [N] [as NAME] [shots N] [basis B] [run=false]
```

The optional items above are templates, not literal brackets. Built-in circuit presets define and queue a run by default: 1,024 shots in Z. `run=false` defines/selects without executing. Bell uses exactly two qubits; GHZ/QFT default to 29 and accept widths 1..298, subject to factor/memory limits on execution. These are different from the 241 q.preset catalogue sequences. Options accept either key value or key=value; unknown/duplicate keys fail.

```
q.circuit preset bell 2 as guide_bell_preset run=false
q.circuit show
q.circuit run shots 256 basis z
```

Without `as NAME`, native default names are bell, ghz-N, and qft-N. Reusing a default preset name can replace that program. Use `as` with a fresh name to make the destination explicit; an existing isolated name is refused.

Register size	Sampling behavior	Readout
1..29	Prepare/replay a fresh shot register and collapse its readout for each shot. The retained output is premeasurement.	x, y, z
30..298	Prepare/replay once, then make a joint Z draw within each factor for every independent shot.	z only

Wide reports identify `samplingMode=prepared-factor-Born-Z`. Correlations within each factor are preserved. One uniform draw is consumed per factor per shot, including deterministic factors. Stored amplitudes are retained rather than collapsed by this sampler.

Wide register example

```
q.circuit new guide_wide 298
q.circuit add h 297
q.circuit add cx 297 0
q.circuit run shots 256 basis z
q.circuit status
```

Only q297 and q0 become correlated; the other qubits stay zero. The two possible outcomes are the all-zero 298-bit string and the string with only its first and last bits set. Logical width is 298, while the correlated factor is small.

Repeated runs consume additional seed-counter positions, so the next histogram need not match the previous one exactly. Use saved program, source hashes, seeds, counters, and samplingMode when comparing recorded runs.

REFERENCE 11

10 / Tranche command dictionary

A tranche supports direct register manipulation. Its gate and resume operations can be background work; poll status before dependent commands. It has its own preparation, memory budget, seed, and replay journal.

Syntax	Effect
`q.tranche help`	Native tranche help and supported gates.
`q.tranche prepare N zero`	Replace the selected register with |0...0>, N=1..298.
`q.tranche from I`	Snapshot procedural block I: starts at family address 29*I, contains up to 29 addresses.
`q.tranche gate GATE ANGLES QUBITS`	Apply one gate to selected register; local gates also accept all.
`q.tranche state INDEX`	Read one amplitude by decimal basis index.
`q.tranche spin Q`	Read reduced Bloch x/y/z of one qubit.
`q.tranche measure Q basis B shots N`	Independent samples of one qubit without collapse, N=1..100000.
`q.tranche measure Q basis B shots 1 collapse`	One actual collapsing sample; changes register and replay journal.
`q.tranche memory MIB`	Set positive whole-MiB budget, large enough to hold current state.
`q.tranche memory auto`	Set budget from measured headroom; no trailing options.
`q.tranche status`	Current register, factor/storage information, and job.
`q.tranche cancel`	Request active operation cancellation.
`q.tranche resume`	Restore selected saved register if it is not live.
`q.tranche reset`	Reset selected register to zero; retain width/seed/counter, clear binding/journal.
`q.tranche save [dense]`	Save compact replay/checkpoint; dense explicitly requests bounded full-state capture.
`q.tranche release`	Release selection/state and journal; seed/counter are preserved.
`q.tranche export`	Export retained register metadata and replay description.

```
q.tranche prepare 2 zero
q.tranche gate h 0
q.tranche status
q.tranche gate cx 0 1
q.tranche status
q.tranche measure 0 basis z shots 100
```

Wait for each gate job to finish before entering the next line. q.query acts on circuit output, not the tranche. A zero Bloch vector is valid for an entangled qubit. The journal limit is 65536 operations; the 59-qubit coherent-factor limit still applies.

REFERENCE 12

11 / Procedural command dictionary

Syntax	Meaning
`q.prepare grid M L`	Define M*L independent addresses; each axis 1..1e9 and product <=2^53-1. Reset migration and composed matrix.
`q.migrate t T`	Set real T in [0,1]; reset previously composed family gates.
`q.gate NAME ANGLES all`	Compose one shared one-qubit matrix across the described family. See smaller vocabulary in gate dictionary.
`q.state ADDRESS`	Calculate one address's spinor, spin, and probabilities.
`q.measure ADDRESS basis B shots N`	Sample one independent address, x/y/z, N=1..100000.
`q.verify rule`	Audit algebraic normalization/spin rules and current 2x2 matrix unitarity.
`q.verify samples N errors-only`	CPU/OpenCL-GPU comparison on distinct planned addresses.
`q.verify status`	Inspect verification plan, counts, previews, and completion.
`q.verify cancel`	Request verification cancellation.
`q.export descriptor`	Export the formula/parameters, not a giant joint statevector.
`q.status`	Full session status, capability identity, counters, and jobs.
`q.reset`	Restore default grid/t/matrix and current-generation represented credits; retain seeds, measurement counter, lifetime actual work, and circuits.
`q.help`	Top-level native help and common command forms.

```
q.prepare grid 1000 1000
q.migrate t 1
q.gate ry 0.2 all
q.state 12345
q.measure 12345 basis z shots 100
q.verify rule
```

An address satisfies 0 <= ADDRESS < M*L. Coordinates follow `id = ell*M + k`, `k = id mod M`, and `ell = floor(id/M)`. The family describes independent single-qubit states; its address count is not an entangled register width.

Verification means a specific check

Rule verification performs a constant-size algebraic/matrix audit. Sample verification materializes selected predictions on the CPU and a real OpenCL GPU, compares implementations and normalization/spin identities, and reports checked coverage. Unsampled floating-point values are not individually verified.

Direct sample requests accept positive decimal integers up to 1,024 digits. The plan clamps coverage to the finite grid; work uses bounded batches and plans above 16,384 samples run asynchronously. Folder verification has its own ceiling. Represented counters describe model equivalents; actual CPU/GPU counters describe performed calculations.

REFERENCE 13

12 / Query the completed circuit

Queries require retained output for the selected program. A run must finish first. State changes and restarts may require rerunning. Bounded prefix dumps and single-qubit reductions can address 298 logical qubits; full joint analysis is bounded to 16 qubits / 65536 amplitudes.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

`q.query amplitudes`

Return complex amplitudes in ascending basis-index order, with an explicit q0-LSB convention.

Syntax: `q.query amplitudes [max=VALUE] [start=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Preset context: `q.preset show qc-154`.

```
q.query amplitudes max=16
```

`q.query probabilities`

Return ideal probabilities separately from shot counts.

Syntax: `q.query probabilities [max=VALUE] [start=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Preset context: `q.preset show qc-155`.

```
q.query probabilities max=16
```

`q.query expectation`

Compute a Pauli-string expectation.

Syntax: `q.query expectation pauli=VALUE`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Preset context: `q.preset show qc-160`.

```
q.query expectation pauli="Z0 Z1"
```

q.query stabilizers

Verify supplied stabilizers by eigenvalue and commutation checks.

Syntax: `q.query stabilizers generators=VALUE`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Preset context: `q.preset show qc-166`.

```
q.query stabilizers generators=["X0 X1","Z0 Z1"]
```

q.query fidelity

Return squared pure-state overlap of the selected coherent output with the supplied normalized target vector.

Syntax: `q.query fidelity state=VALUE [convention=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Target length must match the complete register; vector-input bound is 4096.

```
q.query fidelity state=[1/sqrt(2),0,0,1/sqrt(2)] convention=squared
```

q.query concurrence

Compute pure two-qubit concurrence 2|a00 a11 - a01 a10| for the complete selected two-qubit register.

Syntax: `q.query concurrence qubits=VALUE`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Requires n=2 and both qubits. Mixed reduced-state concurrence is unavailable. Preset context: `q.preset show qc-163`.

```
q.query concurrence qubits=[0,1]
```

q.query reduced-density

Compute a partial trace, preserving coherences on the selected subsystem.

Syntax: `q.query reduced-density qubits=VALUE`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. qubits contains 1..4 distinct indices. A single-qubit request uses native spin and supports wide registers; multiple-qubit requests use full joint analysis. First listed qubit is least significant in local outcomes. Preset context: `q.preset show qc-157`.

```
q.query reduced-density qubits=[0]
```

q.query purity

Return reduced-state purity, not the purity of the entire pure state.

Syntax: `q.query purity qubits=VALUE`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. qubits contains 1..4 distinct indices. A single-qubit request uses native spin and supports wide registers; multiple-qubit requests use full joint analysis. First listed qubit is least significant in local outcomes. Preset context: `q.preset show qc-158`.

```
q.query purity qubits=[0]
```

q.query negativity

Compute pure-state partition negativity from the Schmidt spectrum, equivalent to (||rho^(T_A)||_1 - 1)/2.

Syntax: `q.query negativity partition=VALUE [convention=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. Partition contains 1..4 qubits and must leave a nonempty complement; a one-qubit partition uses native spin and can operate on wide registers. Preset context: `q.preset show qc-164`.

```
q.query negativity partition=[0] convention=trace-norm-minus-one-divided-by-two
```

q.query marginal

Sum joint probabilities over unselected qubits.

Syntax: `q.query marginal qubits=VALUE [basis=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. qubits contains 1..4 distinct indices. A single-qubit request uses native spin and supports wide registers; multiple-qubit requests use full joint analysis. First listed qubit is least significant in local outcomes. Preset context: `q.preset show qc-156`.

```
q.query marginal qubits=[0] basis=z
```

q.query entropy

Report entropy of the reduced state; identify its use as an entanglement entropy for a globally pure state.

Syntax: `q.query entropy qubits=VALUE [base=VALUE]`

Requires the selected circuit's completed retained coherent premeasurement output. Full joint analyses require ≤16 qubits/65536 amplitudes; bounded dumps and one-qubit reductions can use 1-298 logical qubits. Does not sample measurements. qubits contains 1..4 distinct indices. A single-qubit request uses native spin and supports wide registers; multiple-qubit requests use full joint analysis. First listed qubit is least significant in local outcomes. Preset context: `q.preset show qc-159`.

```
q.query entropy qubits=[0] base=2
```

REFERENCE 14

13 / State and matrix calculator dictionary

These commands calculate from explicit input values. They do not use the selected circuit or modify a register. Most state inputs must already have norm one; q.math normalize is the explicit correction.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.math normalize

Divide by the Euclidean norm; reject a zero vector.

Syntax: `q.math normalize amplitudes=VALUE`

Preset context: `q.preset show qc-132`.

```
q.math normalize amplitudes=[1,i]
```

q.math norm

Return Euclidean norm, squared norm and a normalized flag without changing the input.

Syntax: `q.math norm amplitudes=VALUE`

Preset context: `q.preset show qc-133`.

```
q.math norm amplitudes=[1/sqrt(3),sqrt(2/3)]
```

q.math probabilities

Return the squared magnitude of each amplitude.

Syntax: `q.math probabilities amplitudes=VALUE`

Preset context: `q.preset show qc-134`.

```
q.math probabilities amplitudes=[sqrt(3)/2,i/2]
```

q.math bloch

Use x=2Re(conj(alpha)*beta), y=2Im(conj(alpha)*beta), z=|alpha|^2-|beta|^2.

Syntax: `q.math bloch amplitudes=VALUE`

Exactly two amplitudes. Preset context: `q.preset show qc-135`.

```
q.math bloch amplitudes=[1/sqrt(2),i/sqrt(2)]
```

q.math density

Form |psi><psi| with complex conjugation.

Syntax: `q.math density amplitudes=VALUE`

At most 16 amplitudes for density-matrix output. Preset context: `q.preset show qc-138`.

```
q.math density amplitudes=[1/sqrt(2),i/sqrt(2)]
```

q.math canonical-phase

Create a display representative with its first nonzero amplitude real-positive, while returning the original amplitudes and removed phase.

Syntax: `q.math canonical-phase amplitudes=VALUE [preserve-original=VALUE]`

preserve-original can only be true; this is display canonicalization. Preset context: `q.preset show qc-153`.

```
q.math canonical-phase amplitudes=[i/sqrt(2),-1/sqrt(2)] preserve-original=true
```

q.math from-bloch

Return cos(theta/2)|0>+exp(i phi)sin(theta/2)|1>.

Syntax: `q.math from-bloch theta=VALUE phi=VALUE`

theta in [0,pi]; returns two normalized amplitudes. Preset context: `q.preset show qc-136`.

```
q.math from-bloch theta=pi/3 phi=pi/4
```

q.math validate-bloch

Check radius <=1. A vector strictly inside the ball represents a mixed state.

Syntax: `q.math validate-bloch x=VALUE y=VALUE z=VALUE`

Preset context: `q.preset show qc-137`.

```
q.math validate-bloch x=0.2 y=0.3 z=0.4
```

q.math validate-density

Check Hermiticity, trace 1, and positive semidefiniteness.

Syntax: `q.math validate-density rho=VALUE`

Preset context: `q.preset show qc-139`.

```
q.math validate-density rho=[[0.7,0],[0,0.3]]
```

q.math purity

Compute trace(rho^2).

Syntax: `q.math purity rho=VALUE`

Preset context: `q.preset show qc-141`.

```
q.math purity rho=[[0.7,0],[0,0.3]]
```

q.math entropy

Compute von Neumann entropy -Σ λ log_base λ from a valid density matrix; base defaults to 2.

Syntax: `q.math entropy rho=VALUE [base=VALUE]`

Preset context: `q.preset show qc-142`.

```
q.math entropy rho=[[0.5,0],[0,0.5]] base=2
```

q.math overlap

Return the complex inner product <a|b> of two already normalized vectors. Use fidelity for its squared magnitude.

Syntax: `q.math overlap a=VALUE b=VALUE [convention=VALUE]`

Vector dimensions must match. Preset context: `q.preset show qc-143`.

```
q.math overlap a=[1,0] b=[1/sqrt(2),1/sqrt(2)]
```

q.math fidelity

Define squared fidelity for pure states as |<a|b>|^2.

Syntax: `q.math fidelity a=VALUE b=VALUE [convention=VALUE]`

Vector dimensions must match; squared convention. Preset context: `q.preset show qc-144`.

```
q.math fidelity a=[1/sqrt(2),1/sqrt(2)] b=[1/sqrt(2),-1/sqrt(2)] convention=squared
```

q.math trace-distance

Use D=norm(rho-sigma,trace)/2.

Syntax: `q.math trace-distance rho=VALUE sigma=VALUE`

Preset context: `q.preset show qc-145`.

```
q.math trace-distance rho=[[1,0],[0,0]] sigma=[[0,0],[0,1]]
```

q.math gate-matrix

Return H and verify H dagger H=I.

Syntax: `q.math gate-matrix gate=VALUE [angles=VALUE]`

Angle count must match the gate; explicit output at most four operands. Preset context: `q.preset show qc-146`.

```
q.math gate-matrix gate=h
```

q.math compare-gates

Keep global phases in the matrices and explain equivalence.

Syntax: `q.math compare-gates p=VALUE rz=VALUE [up-to-global-phase=VALUE]`

Compares P and RZ with optional global-phase equivalence. CP and CRZ generally differ because controlled-branch phase is observable. Preset context: `q.preset show qc-147`.

```
q.math compare-gates p=pi/3 rz=pi/3 up-to-global-phase=true
```

q.math unitary

Evaluate U dagger U-I rather than relying on gate names.

Syntax: `q.math unitary matrix=VALUE [tolerance=VALUE]`

Preset context: `q.preset show qc-148`.

```
q.math unitary matrix=[[0,1],[1,0]] tolerance=1e-12
```

q.math tensor

Return H tensor I2; the left factor acts on q1 under q0-LSB ordering.

Syntax: `q.math tensor left=VALUE right=VALUE`

Output matrix at most 16x16. Preset context: `q.preset show qc-149`.

```
q.math tensor left=H right=I2
```

q.math commutator

Use [A,B]=AB-BA.

Syntax: `q.math commutator a=VALUE b=VALUE`

Preset context: `q.preset show qc-150`.

```
q.math commutator a=X b=Z
```

q.math eigen

Compute eigenvalues of a supplied Hermitian matrix. For a 2×2 matrix, also return normalized eigenvectors.

Syntax: `q.math eigen operator=VALUE`

Hermitian matrices only; higher dimensions return eigenvalues without eigenvectors. Preset context: `q.preset show qc-151`.

```
q.math eigen operator=X
```

q.math expectation

Return the expectation <psi|operator|psi> of the supplied observable.

Syntax: `q.math expectation state=VALUE operator=VALUE`

Preset context: `q.preset show qc-152`.

```
q.math expectation state=[sqrt(3)/2,1/2] operator=Z
```

REFERENCE 15

14 / Resource planning dictionary

These are estimates under declared representations and conventions. Calculator sampling shots can be as large as 1e15; circuit/measurement execution remains bounded to 100000 shots.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.resource memory

Compute raw complex128 payload: 16×2^n bytes for a statevector, or 16×4^n for a density matrix/unitary; expose relevant working peaks.

Syntax: `q.resource memory n=VALUE representation=VALUE [dtype=VALUE] [units=VALUE]`

n=0..1024 is arithmetic range, not native runnable circuit width. dtype=complex128, units=binary only. Raw 29-qubit statevector = 8 GiB before work buffers. Preset context: `q.preset show qc-200`.

```
q.resource memory n=29 representation=statevector dtype=complex128 units=binary
```

q.resource max-n

Compute the mathematical upper bound and warn that a runnable allocation needs overhead.

Syntax: `q.resource max-n memory-mib=VALUE representation=VALUE [dtype=VALUE] [overhead-mib=VALUE]`

Preset context: `q.preset show qc-203`.

```
q.resource max-n memory-mib=512 representation=statevector dtype=complex128 overhead-mib=0
```

q.resource factor-memory

Estimate sum(16*2^block-size), excluding descriptors and overhead.

Syntax: `q.resource factor-memory block-sizes=VALUE [dtype=VALUE]`

Exact independent factor assumption; entangling gates can merge blocks. Raw sparse/allocator overhead excluded. Preset context: `q.preset show qc-204`.

```
q.resource factor-memory block-sizes=[1,1,2] dtype=complex128
```

q.resource tranche-plan

Compute chunk count and final remainder; explicitly forbid interpreting the chunks as one jointly entangled register.

Syntax: `q.resource tranche-plan addresses=VALUE width=VALUE [semantics=VALUE]`

width remains 1..29 for independent procedural address groups. Preset context: `q.preset show qc-207`.

```
q.resource tranche-plan addresses=500000000000000 width=29 semantics=independent-groups
```

q.resource sampling

Estimate sqrt(p(1-p)/shots) from the declared probability and hypothetical shot count. This read-only arithmetic executes no measurements.

Syntax: `q.resource sampling probability=VALUE shots=VALUE [metric=VALUE]`

Preset context: `q.preset show qc-208`.

```
q.resource sampling probability=0.5 shots=1024 metric=standard-error
```

q.resource shots

Use N>=ln(2/delta)/(2 epsilon^2); identify the bound and assumptions.

Syntax: `q.resource shots epsilon=VALUE delta=VALUE [bound=VALUE]`

epsilon and delta must lie strictly in (0,1). Returned required shots may exceed the runtime 100000-shot limit. Preset context: `q.preset show qc-209`.

```
q.resource shots epsilon=0.01 delta=0.05 bound=hoeffding-two-sided
```

q.resource grover

Report approx pi/4 sqrt(N/M) oracle calls, separating oracle implementation cost.

Syntax: `q.resource grover database-size=VALUE marked=VALUE [formula=VALUE]`

Preset context: `q.preset show qc-210`.

```
q.resource grover database-size=1048576 marked=1 formula=asymptotic
```

q.resource qec

Treat overhead as an assumption, not a hardware benchmark.

Syntax: `q.resource qec logical-qubits=VALUE physical-per-logical=VALUE [model=VALUE]`

Preset context: `q.preset show qc-211`.

```
q.resource qec logical-qubits=29 physical-per-logical=1000 model=user-supplied-overhead
```

q.resource headroom / q.resource auto

Read momentary Windows available RAM and estimate usable peak memory / maximum planned dense width after reserves. q.resource auto reports the same plan; neither applies a budget to the register.

Syntax: `q.resource headroom [reserve-mib=VALUE] [overhead-mib=VALUE] [peak-factor=VALUE] [representation=VALUE] [n=VALUE]`
`q.resource auto [reserve-mib=VALUE] [overhead-mib=VALUE] [peak-factor=VALUE] [representation=VALUE] [n=VALUE]`

Defaults: OS reserve 2048 MiB, extra overhead 256 MiB, peak-factor 3. OS reserve minimum 512 MiB; peak-factor 1..8. Momentary headroom can change.

```
q.resource headroom n=29 representation=statevector
q.resource auto
```

REFERENCE 16

15 / Physics and model dictionary

Small numerical models and educational comparisons use the explicit conventions shown in the examples. Their successful calculation does not establish a device or experimental result.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.model spectrum

Build a small explicit Pauli Hamiltonian matrix and return its eigenvalues and the matrix.

Syntax: `q.model spectrum hamiltonian=VALUE n=VALUE`

Independent supplied-model calculation; does not mutate the selected circuit or measure shots. Auxiliary limits apply rather than the 298-qubit circuit width. n=1..4; ≤64 explicit real coefficient*Pauli terms. Hamiltonian fields contain data, not terminal macros. Preset context: `q.preset show qc-212`.

```
q.model spectrum hamiltonian="-1*Z0" n=1
```

q.model evolve

Use U=exp(-iHt/hbar).

Syntax: `q.model evolve n=VALUE initial=VALUE hamiltonian=VALUE time=VALUE [hbar=VALUE]`

Independent supplied-model calculation; does not mutate the selected circuit or measure shots. Auxiliary limits apply rather than the 298-qubit circuit width. One qubit only (n=1). hbar>0; default 1; initial vector already normalized. Preset context: `q.preset show qc-215`.

```
q.model evolve n=1 initial=[1/sqrt(2),1/sqrt(2)] hamiltonian="0.5*Z0" time=pi/2 hbar=1
```

q.model ising

Count each coupling once and state the exact energy convention.

Syntax: `q.model ising h=VALUE J=VALUE [energy=VALUE] [spins=VALUE]`

Independent supplied-model calculation; does not mutate the selected circuit or measure shots. Auxiliary limits apply rather than the 298-qubit circuit width. At most 12 spins; h real and J real symmetric with zero diagonal. Bit 0 maps to spin +1; bit 1 to -1. Each i<j coupling counted once. Preset context: `q.preset show qc-213`.

```
q.model ising h=[0,0] J=[[0,-1],[-1,0]] energy="sum_i h_i s_i + sum_i<j J_ij s_i s_j"
  spins=plus-minus-one
```

q.model qubo-to-ising

State the full-matrix quadratic convention and include the constant offset.

Syntax: `q.model qubo-to-ising Q=VALUE [objective=VALUE] [mapping=VALUE]`

Independent supplied-model calculation; does not mutate the selected circuit or measure shots. Auxiliary limits apply rather than the 298-qubit circuit width. Q real and square ≤16×16. Implements full-matrix x^TQx with x=(1-s)/2, combining asymmetric off-diagonal entries. Preset context: `q.preset show qc-214`.

```
q.model qubo-to-ising Q=[[1,-1],[-1,1]] objective="x_transpose_Q_x" mapping="x=(1-s)/2"
```

q.model mbqc

Return each branch probability, output state, and Pauli byproduct.

Syntax: `q.model mbqc initial=VALUE [resource=VALUE] [entangler=VALUE] [measurement=VALUE] [branch=VALUE]`

Independent supplied-model calculation; does not mutate the selected circuit or measure shots. Auxiliary limits apply rather than the 298-qubit circuit width. One-bit teleportation with plus ancilla, CZ entangling, X input measurement. Returns both exact branches, each p=1/2, without sampled outcomes. Preset context: `q.preset show qc-217`.

```
q.model mbqc initial=[sqrt(3)/2,1/2] resource="one plus ancilla" entangler=CZ measurement="X on input"
  branch=all
```

q.model compare

Explain ideal model equivalence separately from practical device universality and performance.

Syntax: `q.model compare models=VALUE`

Documented theory entry; accepts only the demonstrated conventions/lists. No numerical hardware/anyon backend is supplied. Preset context: `q.preset show qc-216`.

```
q.model compare models=[gate-based,closed-system-adiabatic,open-system-annealing]
```

q.model compare-codes

Discuss model assumptions; require sources for quantitative device claims.

Syntax: `q.model compare-codes families=VALUE assumptions=VALUE`

Documented theory entry; accepts only the demonstrated conventions/lists. No numerical hardware/anyon backend is supplied. Preset context: `q.preset show qc-218`.

```
q.model compare-codes families=[surface-code,qldpc]
  assumptions=[connectivity,decoder,noise-model,rate,distance]
```

q.model topology

Explain that braiding unitaries require an explicitly named anyon theory; do not invent a physical gate matrix.

Syntax: `q.model topology [braid=VALUE] [anyon-model=VALUE] [action=VALUE]`

Documented theory entry; accepts only the demonstrated conventions/lists. No numerical hardware/anyon backend is supplied. Preset context: `q.preset show qc-219`.

```
q.model topology braid=example anyon-model=unspecified action=explain
```

REFERENCE 17

16 / Lesson dictionary

Lessons return explanations. Fixed comparison lists encode the supported teaching cases. q.lesson pqc returns a dated source snapshot; it does not browse or verify current standards live.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.lesson amplitude-vs-probability

Use this state to explain normalization, phase, and the Born rule.

Syntax: `q.lesson amplitude-vs-probability example=VALUE`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-220`.

```
q.lesson amplitude-vs-probability example=[1/sqrt(2),i/sqrt(2)]
```

q.lesson entanglement

Compare factorability, marginals, and joint correlations.

Syntax: `q.lesson entanglement compare=VALUE`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-221`.

```
q.lesson entanglement compare=["plus tensor plus","Bell phi plus"]
```

q.lesson complexity

Separate established inclusions from open relations; do not claim BQP!=NP is proved.

Syntax: `q.lesson complexity classes=VALUE distinguish=VALUE`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-222`.

```
q.lesson complexity classes=[P,BPP,BQP,PSPACE,NP] distinguish=[proven,conjectured]
```

q.lesson parallelism

Explain that measuring does not reveal every value in a superposition.

Syntax: `q.lesson parallelism [example=VALUE]`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-223`.

```
q.lesson parallelism example=grover
```

q.lesson stabilizer

Explain efficient stabilizer simulation and why entanglement does not establish speedup.

Syntax: `q.lesson stabilizer assumptions=VALUE`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-224`.

```
q.lesson stabilizer assumptions=[stabilizer-input,clifford-gates,pauli-measurements]
```

q.lesson magic

Show T|0>=|0>; avoid universally applying odd-dimensional Wigner-positivity claims to qubits.

Syntax: `q.lesson magic caveats=VALUE`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-225`.

```
q.lesson magic
  caveats=[input-dependent-T,qubit-Wigner-convention,nonclifford-not-sufficient-for-advantage]
```

q.lesson algorithms

Distinguish factoring/discrete-log algorithms from unstructured-search amplitude amplification.

Syntax: `q.lesson algorithms compare=VALUE [scope=VALUE]`

Returns a documented educational explanation and any bounded example arithmetic; executes no circuit or measurement. Preset context: `q.preset show qc-226`.

```
q.lesson algorithms compare=[shor,grover] scope=toy-calculator
```

q.lesson pqc

Return the installed explanatory snapshot for FIPS 203/204/205 and links to their primary source pages.

Syntax: `q.lesson pqc standards=VALUE [mode=VALUE]`

Source snapshot date 2026-10-09. liveNetworkLookupPerformed=false: mode=verify-current-primary-sources is a documented fixed mode, not live browsing. Preset context: `q.preset show qc-227`.

```
q.lesson pqc standards=[FIPS203,FIPS204,FIPS205] mode=verify-current-primary-sources
```

REFERENCE 18

17 / Noise and mitigation dictionary

These commands operate on supplied small states/circuits. A noise channel may return a mixed density matrix. Parameters depend on the channel; p, gamma, and lambda are different conventions.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.noise apply

Use E(rho)=(1-p)rho+p XrhoX.

Syntax: `q.noise apply [state=VALUE] [rho=VALUE] channel=VALUE [p=VALUE] [gamma=VALUE] [lambda=VALUE] [convention=VALUE]`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. Exactly one state or rho; one qubit. Channels: bit-flip/phase-flip/depolarizing with p, amplitude-damping with gamma, dephasing with lambda; parameter in [0,1]. Preset context: `q.preset show qc-168`.

```
q.noise apply state=[1,0] channel=bit-flip p=0.1
```

q.noise circuit

Apply a local X channel to q0 after preparing Bell phi-plus.

Syntax: `q.noise circuit n=VALUE input=VALUE gates=VALUE targets=VALUE [independent=VALUE] channel=VALUE [p=VALUE] [gamma=VALUE] [lambda=VALUE] [convention=VALUE]`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. n=1..4; prepare gates first, then apply the selected local channel to targets. No interleaved channel schedule. independent, if supplied, must be true. Preset context: `q.preset show qc-174`.

```
q.noise circuit n=2 input=0 gates="h 0;cx 0 1" channel=bit-flip p=0.1 targets=[0]
```

q.noise t1

Use p1(t)=p1(0) exp(-t/T1), with matching units.

Syntax: `q.noise t1 excited-probability=VALUE time-us=VALUE t1-us=VALUE`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. time-us≥0 and t1-us>0. Formula is a specified phenomenological relaxation model. Preset context: `q.preset show qc-176`.

```
q.noise t1 excited-probability=1 time-us=20 t1-us=100
```

q.noise t2

Use coherence(t)=coherence(0) exp(-t/T2) in a specified phenomenological model.

Syntax: `q.noise t2 coherence=VALUE time-us=VALUE t2-us=VALUE`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. time-us≥0 and t2-us>0; |coherence|≤1/2. Formula is a specified phenomenological model. Preset context: `q.preset show qc-177`.

```
q.noise t2 coherence=0.5 time-us=20 t2-us=100
```

q.noise readout

Multiply the column-stochastic confusion matrix by the true probability column.

Syntax: `q.noise readout probabilities=VALUE confusion=VALUE [orientation=VALUE]`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. Confusion matrix must be column-stochastic; rows=observed, columns=true. Returns a forward model, not readout inversion. Preset context: `q.preset show qc-178`.

```
q.noise readout probabilities=[0.75,0.25] confusion=[[0.98,0.03],[0.02,0.97]]
  orientation=rows-observed-columns-true
```

q.noise zne

Fit a line and evaluate at zero noise. Label this an extrapolated estimate rather than a recovered exact result.

Syntax: `q.noise zne scales=VALUE expectations=VALUE [model=VALUE]`

Supplied state/data analysis, separate from procedural family and selected circuit. Channels use physical density matrices; does not sample measurement shots. 2..64 matching values, positive scales and at least two distinct scales. linear model only; intercept is an extrapolated estimate. Preset context: `q.preset show qc-179`.

```
q.noise zne scales=[1,2,3] expectations=[0.8,0.7,0.6] model=linear
```

REFERENCE 19

18 / Error-correction dictionary

Ideal educational demonstrations require an explicit encoding/recovery model. They do not establish hardware fault tolerance. Parameters arithmetic uses a supplied number of independent stabilizers; it does not prove independence.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.qec validate

Check commuting Hermitian Pauli generators, independence and absence of -I.

Syntax: `q.qec validate stabilizers=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Phase-aware Pauli consistency, commutation and rank analysis; inferred n includes every index 0..highest. No code-distance proof or tableau circuit simulation. Preset context: `q.preset show qc-185`.

```
q.qec validate stabilizers=["X0 X1","Z0 Z1"]
```

q.qec parameters

Compute k=n-rank from the supplied physical-qubit count and number of independent stabilizers.

Syntax: `q.qec parameters n=VALUE independent-stabilizers=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. The user asserts commuting, independent, consistent checks. This action does not examine generator strings or determine distance. Preset context: `q.preset show qc-187`.

```
q.qec parameters n=7 independent-stabilizers=6
```

q.qec correction-budget

Use t=floor((d-1)/2) for arbitrary unknown Pauli errors.

Syntax: `q.qec correction-budget distance=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Preset context: `q.preset show qc-188`.

```
q.qec correction-budget distance=5
```

q.qec compare

Explain operations, assumptions and what each method estimates or protects.

Syntax: `q.qec compare [mitigation=VALUE] [correction=VALUE]`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Preset context: `q.preset show qc-189`.

```
q.qec compare mitigation=zne correction=stabilizer-code
```

q.qec repetition3 encode

Map alpha|0>+beta|1> to alpha|000>+beta|111>.

Syntax: `q.qec repetition3 encode state=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Preset context: `q.preset show qc-181`.

```
q.qec repetition3 encode state=[1/sqrt(2),1/sqrt(2)]
```

q.qec repetition3 simulate

Compute syndrome, recovery and decoded state; explicitly supply the one-error noise assumption.

Syntax: `q.qec repetition3 simulate state=VECTOR (error=ERROR | errors=LIST) [recover=true|false]`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Supply exactly one error or errors. Bit repetition accepts I/X0/X1/X2; phase repetition accepts I/Z0/Z1/Z2; ≤32 specified errors. Preset context: `q.preset show qc-182`.

```
q.qec repetition3 simulate state=[sqrt(3)/2,1/2] error=X1 recover=true
```

q.qec phase-repetition3 encode

Encode one logical qubit into the H-on-all-qubits conjugate of the three-qubit repetition code.

Syntax: `q.qec phase-repetition3 encode state=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold.

```
q.qec phase-repetition3 encode state=[1/sqrt(2),i/sqrt(2)]
```

q.qec phase-repetition3 simulate

Conjugate bit-flip encoding and syndrome checks by H on all code qubits.

Syntax: `q.qec phase-repetition3 simulate state=VECTOR (error=ERROR | errors=LIST) [recover=true|false]`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Supply exactly one error or errors. Bit repetition accepts I/X0/X1/X2; phase repetition accepts I/Z0/Z1/Z2; ≤32 specified errors. Preset context: `q.preset show qc-184`.

```
q.qec phase-repetition3 simulate state=[1/sqrt(2),i/sqrt(2)] error=Z2 recover=true
```

q.qec repetition3 syndrome-table

Tabulate signed stabilizer outcomes for one-X-error cases.

Syntax: `q.qec repetition3 syndrome-table errors=VALUE stabilizers=VALUE`

Bounded ideal code demonstration or algebra/arithmetic. Does not establish hardware fault tolerance, decoder performance or an error threshold. Preset context: `q.preset show qc-180`.

```
q.qec repetition3 syndrome-table errors=[I,X0,X1,X2] stabilizers=["Z0 Z1","Z1 Z2"]
```

REFERENCE 20

19 / Compilation and algorithm dictionary

Compilation commands compare or analyze explicit bounded gate scripts. Semicolons belong inside a quoted script field. Gate-script angles use restricted real expressions.

Dictionary notation: required fields appear directly; optional fields appear in [brackets]. VALUE means an action-specific scalar, text, list, vector, or matrix. Examples supply concrete forms. Unknown fields and duplicate fields are rejected. These calculator namespaces have no help subaction; consult this dictionary, q.help, and the preset catalogue.

q.compile universality

Explain density in single-qubit unitaries with an entangling gate; do not claim exact representation of every continuous gate.

Syntax: `q.compile universality set=VALUE [semantics=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. Preset context: `q.preset show qc-199`.

```
q.compile universality set=[h,t,cx] semantics=approximate
```

q.compile identity

Compare full matrices, including phase.

Syntax: `q.compile identity left=VALUE right=VALUE n=VALUE [equivalence=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. n=1..4; compare numerical full matrices under equivalence=exact (default) or global-phase. Preset context: `q.preset show qc-190`.

```
q.compile identity left="h 0;x 0;h 0" right="z 0" n=1 equivalence=exact
```

q.compile simplify

Remove supported adjacent inverse pairs while retaining global phase; does not reorder noncommuting gates.

Syntax: `q.compile simplify n=VALUE gates=VALUE [preserve-global-phase=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. n=1..29; ≤256 instructions in ≤12000-character gate field. all expands local instructions. Auxiliary analysis stays 29-wide. Preset context: `q.preset show qc-193`.

```
q.compile simplify n=1 gates="h 0;h 0;t 0;tdg 0;ry 0.2 0;ry -0.2 0" preserve-global-phase=true
```

q.compile count

Count logical operations and non-Clifford phase gates without hardware assumptions.

Syntax: `q.compile count n=VALUE gates=VALUE [counting=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. n=1..29; ≤256 instructions in ≤12000-character gate field. all expands local instructions. Auxiliary analysis stays 29-wide. Preset context: `q.preset show qc-194`.

```
q.compile count n=2 gates="h 0;t 0;cx 0 1;tdg 1" counting=logical
```

q.compile depth

Schedule disjoint operations while preserving per-qubit dependencies.

Syntax: `q.compile depth n=VALUE gates=VALUE [connectivity=VALUE] [scheduling=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. n=1..29; ≤256 instructions in ≤12000-character gate field. all expands local instructions. Auxiliary analysis stays 29-wide. Preset context: `q.preset show qc-195`.

```
q.compile depth n=3 gates="h 0;h 1;cx 0 2;z 1" connectivity=all-to-all
  scheduling=earliest-dependencies
```

q.compile classify

Classify supplied gate structure for eligibility under a computational-basis/stabilizer model.

Syntax: `q.compile classify n=VALUE gates=VALUE [input=VALUE]`

Restricted gate-script analysis, separate from editing/executing the selected circuit; no measurement draws. n=1..29; ≤256 instructions in ≤12000-character gate field. all expands local instructions. Auxiliary analysis stays 29-wide. A classification does not install a stabilizer backend or demonstrate a speedup. Preset context: `q.preset show qc-196`.

```
q.compile classify n=2 gates="h 0;s 1;cx 0 1" input=computational-basis
```

q.algorithm phase-estimation

Use counting qubits q0,q1,q2 and eigenstate |1> on q3. The P(3*pi/4) eigenvalue is exp(2*pi*i*3/8); controlled powers and an inverse QFT recover the three-bit phase.

Syntax: `q.algorithm phase-estimation counting-qubits=VALUE unitary=VALUE eigenstate=VALUE [power-map=VALUE] [inverse-qft=VALUE]`

Bounded exact demonstration from supplied arguments, separate from the selected named circuit; reports ideal probabilities rather than sampled counts. counting-qubits=1..8; unitary must be P(angle) on target q=counting-qubits; eigenstate must be computational |0> or |1>. Explicit controlled powers / inverse-QFT with final swaps; not arbitrary-unitary phase estimation or Shor factoring. Preset context: `q.preset show qc-241`.

```
q.algorithm phase-estimation counting-qubits=3 unitary="p 2.356194490192345 3" eigenstate=[0,1]
  power-map="qj-controls-U^(2^j)" inverse-qft=include-final-swaps
```

Unavailable synthesis

`q.compile synthesize` and catalogue preset qc-198 remain unavailable. The bundled engine lacks the required verified number-theoretic Clifford+T synthesizer. Native preflight reports that prerequisite rather than returning an invented synthesis result.

REFERENCE 21

20 / Memory controls and analysis limits

Limit or control	Meaning
Circuit / zero-prepared tranche width	1..298 logical qubits.
One coherent joint factor	At most 59 qubits, plus actual memory budget.
Direct circuit shots	1..100000; 1024 for built-in preset default.
Circuit instructions	8192 per program; 65536 total saved instructions.
Direct gate angle	Finite magnitude <= 1e12 radians.
Full circuit joint-query materialization	At most 16 qubits / 65536 amplitudes; output dumps max 4096 rows.
Small calculator vectors / matrices	Vectors 1..4096 amplitudes; matrices up to 16x16; operation-specific limits can be smaller.
Explicit gate-matrix output	Up to four operands; mcx7 is outside this output bound.
Circuit capacity	Default 64; adjustable 1..1024.
Tranche replay journal	65536 operations.
Default coherent storage budget	512 MiB; inspect current status because saved/manual settings can differ.

Dense growth

For complex128 statevectors, amplitudes alone cost `16 * 2^n` bytes. A density matrix or full unitary costs `16 * 4^n` bytes. A factor representation can save storage when the actual state permits it; the logical width alone does not determine allocation.

Dense statevector n	Amplitude bytes only
20	16 MiB
25	512 MiB
26	1 GiB
29	8 GiB

```
q.resource headroom
q.resource auto reserve-mib=2048 overhead-mib=128 peak-factor=3
q.circuit memory auto
q.tranche memory auto
```

q.resource auto reports a plan without applying it. Plain memory auto applies the native default plan. For a custom budget, convert usablePeakBytes to whole MiB and enter q.circuit memory MIB or q.tranche memory MIB. Plans reserve at least 512 MiB for the OS; allocations can still fail.

Narrow advanced bounds: q.model spectrum uses 1..4 qubits; q.model evolve exactly one; q.noise circuit up to 4; q.compile identity 1..4; phase-estimation counting-qubits 1..8. These are independent of logical register width.

REFERENCE 22

21 / Attachment command dictionary

Upload through the interface first. The returned attachment ID is required in later commands. Uploading or inspecting preserves source bytes/hash and does not automatically execute gates or fit data.

Syntax	Effect
`q.input help`	Show supported input forms and native help.
`q.input list`	List stored attachments.
`q.input show ID`	Preview source/metadata; source preview capped at 32768 characters.
`q.input table ID`	Preview a CSV table and columns.
`q.input export ID`	Export exact original bytes/source metadata; PDF uses binary download.
`q.input delete ID`	Remove an attachment; interactive action.
`q.input evaluate ID [row R] [base BASE_ID] [name=VALUE ...]`	Evaluate a program/parameter binding without importing it.
`q.input import ID [row R] [base BASE_ID] [name=VALUE ...]`	Import evaluated circuit definition; does not run its gates.
`q.input run ID`	Queue executable command/circuit attachment using declared hints.
`q.input fit ID target=T features=[F,...]`	Fit numeric measurement rows. Optional target-unit and feature-units labels.

Parameter bindings are explicit

```
q.input evaluate CSV_ID row 0 base CIRCUIT_ID
q.input import CSV_ID row 0 base CIRCUIT_ID
q.circuit run shots 1024 basis z
```

Use actual attachment IDs. A parameter CSV needs a separately attached base circuit and zero-based row. Evaluate overrides accept whitespace-free expressions; import overrides require finite number literals. Source circuit/CSV cells can retain expressions. A missing base is not guessed.

Fit measurements

```
q.input table ATTACHMENT_ID
q.input fit ATTACHMENT_ID target=3 features=[1,2] target-unit="degrees C"
```

Select headers by name or zero-based index. Indexes avoid ambiguous numeric names or punctuation/spaces. `feature-units=["percent","hPa"]` must match feature order. Unit labels are supplied, never inferred.

Nested q.input/q.folder actions are refused inside command-file sequences. Text attachment limit 5 MiB; PDF 4 MiB; 32 attachments and 30 MiB combined source bytes. q.input run compilation still caps a source at 4 MiB. Returned fit statistics refer to training rows; no independent holdout is performed.

REFERENCE 23

22 / Program-file grammar

Format	How the parser reads it
.txt / .circuit	Either a supported command sequence or a text circuit. Blank/comment lines are ignored. Use # SPINDOCTOR-SEQUENCE to force a sequence.
.json	commands array, gate-based circuit object, or supported declarative circuit schema.
.qasm / .qasm2	Bounded OpenQASM 2.0 subset: declarations, supported gates, and measurement syntax; not a complete QASM implementation.
.csv	Command CSV, complete reserved gate CSV, measurement table, or parameter CSV; schemas have distinct uses.
.pdf	One supported embedded program attachment or a matching SPINDOCTOR format-marker block. Scans/prose are not guessed.

Explicit command sequence (.txt)

```
# SPINDOCTOR-SEQUENCE
q.circuit new file_bell 2
q.circuit add h 0
q.circuit add cx 0 1
q.circuit run shots 256 basis z
```

Use the marker for incremental packs. Without it, new/input/add-only files may be circuit definitions. Prose, fences, nested input/folder runs, reset/deletion, and unquoted macro separators are refused.

Text circuit (.circuit)

```
qubits 2
shots 256
h 0
cx 0 1
```

JSON command array

```
{"commands":["q.math normalize amplitudes=[1,i]",
             "q.math bloch amplitudes=[1/sqrt(2),i/sqrt(2)]"]}
```

JSON gate circuit

```
{"name":"json_h","qubits":1,"gates":[{"name":"h","qubits":[0]}],
 "shots":256,"basis":"z"}
```

Files: textual UTF-8 / binary PDF, top-level, <=4 MiB. Sequences: 1..128 expanded commands, <=4096 characters each. A queued circuit adds four commands, leaving 124 source gates; import larger circuits directly. Text/JSON/QASM default to one Z shot; gate CSV to 1024.

REFERENCE 24

23 / QASM and PDF input boundaries

A minimal OpenQASM 2.0 example

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[2];
h q[0];
cx q[0],q[1];
measure q -> c;
```

The subset uses exactly one qreg (1..298 qubits), an optional same-width creg, and a symbolic qelib1.inc include before gates. Measurements must be final, complete Z readout with matching indices. Reset, if, custom gate, opaque, snapshot, and mid-circuit measurements are refused. Barrier is a presentation marker. General QASM 3 is not supported.

QASM aliases include u1(angle)=p(angle), u2(phi,lambda)=u(pi/2,phi,lambda), cu1=cp, and cu3(theta,phi,lambda)=cu(theta,phi,lambda,0). Register indices are zero-based. Semicolons are QASM punctuation, not terminal command chaining.

A PDF must identify its executable content

```
SPINDOCTOR-TXT-BEGIN
# SPINDOCTOR-SEQUENCE
q.math probabilities amplitudes=[1/sqrt(2),i/sqrt(2)]
SPINDOCTOR-TXT-END
```

Other matching markers use JSON, QASM, or CSV in place of TXT. A PDF may instead embed one .txt/.csv/.json/.qasm attachment. Keep surrounding narrative outside the marker block. A scanned page is not OCRed into commands. Encrypted PDFs are unsupported; source limit is 4 MiB, decoded streams 2 MiB each and 16 MiB aggregate.

A program file is different from a report

An exported report records results and provenance. Its readable equation or circuit drawing is not automatically an executable input program. Editing a displayed equation draft does not change coefficients, source CSV, authenticated receipts, or a previously computed state.

Round-trip deliberately

For a new calculation, edit the actual source program/data, upload it, inspect its declared interpretation, then explicitly Import, Run, or Fit. Preserve the original source and recorded ID/hash when comparing results.

This expanded guide itself is a prose reference PDF. It is not a folder-run program. Copy the relevant plain command block or use an explicitly supported program-file schema.

REFERENCE 25

24 / CSV dictionary and schemas

CSV schema	Recognizing headers	Explicit action
Command program	Exactly one header: command	q.input run ID or folder execution.
Gate program	Complete reserved set: gate,n,targets,angles	q.input import ID, then circuit run; or q.input run ID.
Measurement data	Ordinary table; select target/features	q.input table ID, then q.input fit ID.
Parameter sweep	Headers match parameters declared in a base circuit	evaluate/import with row and base ID.

Command CSV

```
command
"q.math normalize amplitudes=[1,i]"
"q.math bloch amplitudes=[1/sqrt(2),i/sqrt(2)]"
```

Comma-containing commands must be CSV-quoted. Literal quotes inside a CSV cell are doubled. Command cells must stay single-line; limits are 1..128 commands and 4096 characters each.

Gate CSV

```
gate,n,targets,angles,name,initialBasis,shots,basis
h,2,0,,csv_bell,0,1024,z
cx,,"0,1",,,,,
```

Required columns are gate,n,targets,angles. First gate row declares n; the updated parser accepts 1..298 even though older CSV prose says 1..29. Optional metadata can be blank later or repeat consistently. Defaults are initialBasis=0, shots=1024, basis=z. Conflicts are refused.

targets use control-first operands. Angles accept safe expressions in radians; comma lists such as "[pi/2,0,pi]" need CSV quotes. No-angle gates use an empty cell. Only the complete reserved headers select gate format; partial gate/n headers do not turn a table into a program.

Measurement CSV

```
humidity,pressure,temperature
45,1002,35.2
55,998,33.8
65,1005,32.1
50,1008,34.4
```

Fit limits: 1..64 unique headers, 4096 rows, 1..16 predictors. Selected cells must be finite decimal/scientific values, magnitude <=1e150. Missing/nonnumeric selected cells are excluded with zero-based row/reason; unused text columns do not exclude valid rows.

Need more complete rows than predictors plus intercept; constant/collinear predictors fail. Constant-target R-squared is null. Cells <=4096 characters; linear probability fits may leave [0,1].

REFERENCE 26

25 / Presets, sequences, and folder runs

Syntax	Meaning
`q.preset list`	List 241 catalogue presets; historical observed/proposed labels do not decide current availability.
`q.preset show ID`	Show commands, parameters, prerequisites, and runtime availability.
`q.preset run ID [key=VALUE ...]`	Start one preset sequence with documented parameter overrides.
`q.sequence help`	Show native sequence help.
`q.sequence status`	Inspect the active/recent sequence.
`q.sequence cancel`	Request cancellation and poll until stopped.
`q.sequence export [JOB_ID]`	Export current or specified saved sequence/job description.
`q.folder help`	Show formats, ordering, limits, and verification policy.
`q.folder status`	Observe current folder sequence.
`q.folder cancel`	Request folder cancellation.
`q.folder run path="ABSOLUTE_PATH" ...`	Queue supported top-level files in strict ordinal filename order.

The UI/API submits an edited sequence as an array of ordinary commands. `q.sequence run` is the recorded operation label for that submission; it is not a terminal text command accepting a concatenated program.

Folder options and actual defaults

Option	Default	Effect
path	Required	Absolute local drive path; forward slashes simplify quoted paths.
downloads	true	Request automatic report downloads through the UI.
verify	true	CPU/OpenCL-GPU sample check after each file before verified receipt.
samples	10240	Folder verification count; 1..16384000000.
purge	false	If true, release temporary coherent output after native archive capture.

```
q.folder run path="D:/Codex/inputs" downloads=false verify=false
q.folder status
```

The path above is a template and must exist. verify=false creates explicitly unverified receipts. verify=true needs a working OpenCL GPU and is separate from circuit sampling. purge=true preserves programs, sources, seeds/history, and archived results; live queries may require a new run.

REFERENCE 27

26 / Ordered jobs and repeatable file packs

What a folder job guarantees

Files run serially in ordinal filename order: digits, capitals, underscore, then lowercase. Use fixed-width prefixes such as 01_, 02_, 03_. Enumeration is nonrecursive. The first failing file stops the job; earlier results remain, and later files do not run.

Folder constraint	Bound or behavior
Supported extensions	.txt, .csv, .json, .qasm, .qasm2, .circuit, .pdf
Supported files per job	1..128
Size per file	4 MiB; strict UTF-8 for textual input.
Commands per file	1..128; each <=4096 characters.
Unsupported entries	Skipped; inspect skippedFiles/skippedFileCount.
Path restrictions	Absolute local drive; network paths, junctions/symlinks, and reparse-point ancestors/files refused.
Active sequence	Owns calculation context; use status/help/history/export/cancel and permitted preset inspections.

Multipart circuit construction

Begin each file with `# SPINDOCTOR-SEQUENCE`. The first creates/selects a circuit; middle files append gates; the final runs it. Multiple files still build one circuit state.

```
# 01_define.txt
# SPINDOCTOR-SEQUENCE
q.circuit new pack_demo 2
q.circuit add h 0

# 02_finish.txt (a separate file)
# SPINDOCTOR-SEQUENCE
q.circuit add cx 0 1
q.circuit run shots 256 basis z
```

Keep notes, manifests, and results outside the runnable folder. Repeating new with the same name can fail; use fresh names or manage the existing circuit before rerunning.

Interpret receipts in context

A completed calculation, verification, and downloadable report are separate states. Inspect job phase/error, completed commands/shots, verificationRequired, receiptKind, and receiptReady. verificationPhase, reportPhase, and purgePhase separately report checking, archival, and cleanup. An unverified receipt records that limited scope.

Online-only OneDrive files can be refused; make files locally available or use a plain local folder. Long runs can create large archives. Compact status reports retain child records.

REFERENCE 28

27 / Worked language recipes

A rotated qubit: decimal terminal angle

```
q.circuit new guide_rotate 1
q.circuit add ry 1.5707963267948966 0
q.circuit run shots 256 basis z
q.circuit status
q.circuit spin 0
```

Wait for the run before spin. Ideal RY(pi/2)|0> points along +X and has Z probabilities 1/2,1/2. The calculator form accepts the symbolic angle:

```
q.math gate-matrix gate=ry angles=[pi/2]
```

A GHZ state

```
q.circuit new guide_ghz 3
q.circuit add h 0
q.circuit add cx 0 1
q.circuit add cx 0 2
q.circuit run shots 256 basis z
q.circuit status
q.query probabilities max=8
```

After completion, only 000 and 111 have nonzero ideal probabilities. Each individual qubit has a mixed reduction, even though the whole coherent state is pure.

An identity check using a quoted gate script

```
q.compile identity left="h 0;x 0;h 0" right="z 0" n=1 equivalence=exact
```

Each semicolon separates a gate inside left. The entire line is one calculator command. It checks H X H = Z under the declared equivalence convention.

Estimate sampling uncertainty

```
q.resource sampling probability=0.5 shots=1024 metric=standard-error
q.resource shots epsilon=0.01 delta=0.05 bound=hoeffding-two-sided
```

For p=1/2 and 1024 independent shots, standard error is 0.015625 (1.5625 percentage points). The requested error bound is a planning calculation; it does not run those shots.

REFERENCE 29

28 / Grammar mistakes and their corrections

Problem	Use this form	Why
`q.circuit add ry pi/2 0` typed directly	`q.circuit add ry 1.5707963267948966 0`	Direct gate angles use number literals.
`q.circuit run shots=256 basis=z`	`q.circuit run shots 256 basis z`	This action uses positional labels.
`q.math normalize [1,i]`	`q.math normalize amplitudes=[1,i]`	Calculator fields are named.
`q.query expectation pauli=Z0 Z1`	`q.query expectation pauli="Z0 Z1"`	Spaces need quotes.
`q.circuit add cx all`	`q.circuit add cx 0 1`	Controlled gates need explicit operands.
`q.circuit add cx 0 0`	`q.circuit add cx 0 1`	Operands must be distinct.
`q.circuit add h 2` in a 2-qubit circuit	Use operand 0 or 1, or create a wider circuit.	Qubits are zero-based.
`q.math probabilities amplitudes=[1,1]`	Normalize explicitly, then use normalized amplitudes.	Most state inputs must already have norm one.
`q.math help` or `q.compile help`	Use this dictionary / q.help / q.preset show ID.	No calculator help subaction is installed.
Two terminal commands separated by ;	Submit each as a separate line/array element.	No top-level command macro chaining.
Text file with a ``` code fence	Remove the fences; comment prose with #.	Command input is program text, not Markdown.
298-qubit run in basis x	Use basis z.	Wide prepared-factor sampler currently supports Z.

Avoid silent assumptions

Use explicit **state scope**, **index order**, **basis**, **normalization**, and **completion state**. Treat the returned error as a parser explanation, not a reason to guess a similar-looking command. Read native help for operational families and q.preset show for complete prerequisite sequences.

Commands ending in help exist for q.circuit, q.tranche, q.input, q.sequence, and q.folder. The actual calculator registry enumerates valid actions; a family name alone does not promise a help action.

REFERENCE 30

29 / Reports, results, and provenance

Record	Use it for
TXT	Complete readable record of native command results, metadata, and errors.
PDF	Readable report with equations, circuit diagrams, histograms, and state figures where provided.
CSV	Typed nested values for spreadsheet inspection; distinct from executable input CSV schemas.
Program/descriptor export	Saved definition and state/session metadata; inspect schema before treating it as input.
Seed and counters	Recorded deterministic sampling stream positions and calculation history.
Source/program hash	Identity of input bytes or the declared gate program.
Sampling mode	Distinguishes independent-shot replay from prepared-factor Born Z sampling.
Receipt kind and verification phase	States which verification conditions were met for that owner/job.

Every terminal command archives a record, including errors and status checks. Native archives live in SpinDoctor/workspace under folders including quantum-command-reports, sequence-reports, and sequence-jobs. Interface downloads go to the browser's download location.

Exports read saved results; they do not rerun gates, refit a table, or repeat verification. Inspect requested versus completed shots and job error before drawing conclusions from a partial histogram. q.folder status and q.sequence status archive compact polling metadata; q.status saves the full snapshot.

History commands

Syntax	Effect
`q.history help`	Show history command help.
`q.history list`	List recent report records.
`q.history show ID`	Read one saved command record.
`q.history export ID`	Export the record through the native history action.

Equation editor

Load selected native equations explicitly. The local text editor supports copying/resetting a working draft and identifies its native source revision. Editing that draft does not change the calculation or report receipt. Refit/import/run changed source data to obtain a new native result.

PDF report export has its own bounds (including 512 pages and 32 MiB output). Keep native TXT/CSV records for complete numeric inspection when a visual preview is bounded or truncated.

REFERENCE 31

30 / Separate virtual dashboard language

The Virtual Domain Ledger has its own VXPU text console. These tokens belong there, not in the q.* quantum terminal. Its commands manipulate a compact virtual epoch description and sample selected queries. Their virtual-operation counts do not establish physical quantum or machine-operation throughput.

Token / syntax	Meaning
`help`	List virtual console syntax.
`status`	Current epoch and virtual execution/accounting state.
`registers`	Read registers/status records.
`advance N`	Advance a positive integer number of virtual epochs.
`qflop [N]`	Advance N*100000 epochs; default N=4, positive integer.
`eval LANE [EPOCH]`	Materialize one canonical lane multiply/add pair; omitted EPOCH uses current.
`op ID [EPOCH]`	Resolve one operation ID: even=multiply, odd=add; show its lane pair.
`fma A B C`	Compute FP32-rounded multiply then FP32-rounded add for three integers.
`verify`	Check procedural definition, binding, ranges, counts, and commit chain.
`clear`	Clear console text while retaining device/domain state.

This parser trims input, splits whitespace, lowercases the command token, and accepts at most 2,048 characters and eight tokens. Canonical decimal addresses have no leading zeros except 0. LANE is 0..5*10^24-1; operation ID is 0..10^25-1. Epoch queries require initialEpoch < EPOCH <= currentEpoch; epoch ceiling is 2^32-1. fma integers are -1024..1023.

```
advance 1
eval 0
op 1
fma 13 7 2
status
```

Internal opcodes are a different vocabulary

The generated VCK-1 stream uses `VLOAD`, `VXOR`, `VADDMUL`, `VADDR`, `VSTORE`, `VBRANCH`, `VPHASE`, and `VCOMMIT`. These are internal instruction labels shown in the command engine dashboard; that dashboard does not expose them as a typed terminal language.

Dashboard-specific controls and their state are independent of selected q.circuit/q.tranche registers. Use each dashboard's own help/status and visible controls. The word fma here describes a formula, not a promise that a fused hardware instruction was measured.

REFERENCE 32

31 / Launcher and package-tool vocabulary

These are Windows launcher files, not q.* commands. Use the normal package root containing BOOT.cmd, harness, and cartridge. Saved projection aliases resolve back to that package.

Tool	Purpose
`BOOT.cmd [gif]`	Verify/decode current carrier, refresh program projection, and launch terminal. TIFF default.
`BOOT_SAFE.cmd`, `BOOT_DIRECT.cmd`, `BOOT_LOOPBACK.cmd`	Current aliases for the terminal launcher.
`VERIFY.cmd [gif|tiff|filename]`	Check one specified carrier or both current carriers by default.
`INSPECT.cmd [gif|tiff|filename]`	Print carrier inventory.
`BUILD_HARNESS.cmd`	Confirm harness compilation.
`DIAGNOSE.cmd`	Inspect integrity, required native/UI payloads, and corresponding saved projection hashes.
`RESET_CURRENT.cmd`	Preserve saved data and explain q.reset; does not erase the workspace.
`BENCHMARK_DASHBOARD.cmd`	Refresh measured carrier decode/hash and MVP smoke metrics.
`PACKAGE_TOOLS.ps1 -Mode Smoke`	Bounded carrier smoke inspection; no materialization or model execution.

Carrier benchmarks measure the stated decode/hash/smoke work and its image hash. Compilation/process startup are outside those recorded timings. They do not measure quantum-model execution or represented-operation throughput.

When the browser does not open

Copy the local address printed by the launcher console into the browser. Keep the console open. If carrier/harness files are missing, keep the distribution together and use VERIFY/DIAGNOSE before changing its layout.

The inspected source code is the distribution's source/mvp tree. Existing workspace/projection files may reflect a previously materialized version until BOOT refreshes it. Do not manually substitute historical extracted code when you need the current update identity.

REFERENCE 33

32 / Troubleshooting dictionary

Message or symptom	Interpretation and next step
Circuit name already exists	Use a fresh NAME, or select the saved circuit. Delete only if you intend to discard that definition.
No retained coherent output	Run the selected program and wait; state/query needs matching live output.
A circuit/tranche/sequence job is active	Poll its status; cancellation is a request, so wait for inactive state.
Out of memory / budget exceeded	Inspect resource estimate and headroom; reduce joint factor/program or explicitly choose a viable budget.
Wide register basis refusal	Use Z for a 30..298-qubit circuit.
Operand outside circuit range	Inspect selected name and width. q.migrate does not change either.
Unknown calculator action or field	Use exact action/keys from the dictionary. No generic help subaction or arbitrary extra keys.
Input sequences accept documented commands only	Remove prose, fences, nested input/folder commands, reset/deletion, and top-level chaining.
Text input is empty	Only comments/blanks were supplied; add a real program.
Text circuit requires qubits N	Declare width before gate lines, or force an explicit command sequence marker.
Unable to translate bytes	Save textual input as valid UTF-8.
Reparse-point path refused	Use fully local plain files; online-only OneDrive placeholders or links can be refused.
No supported files found	Check extensions and skipped-file list; enumeration is top-level only.
Verification failed or receipt withheld	Inspect verification phase/error and OpenCL availability; use verify=false only when unverified output is intended.
Preset unavailable (qc-198)	Verified Clifford+T synthesis backend is absent.
Fit rejects predictors	Need enough complete rows, nonconstant predictors, and independent columns.
R-squared is null	A constant target has no defined variance-normalized score.
Repeated folder execution stops at new	Choose new circuit names or explicitly manage collisions before rerun.

Long runs preserve child reports and can consume substantial disk space. Review archived runs before moving/removing obsolete workspace copies. The guide's examples use fresh names to make collisions easy to identify.

REFERENCE 34

33 / Vocabulary dictionary: A-M

Definitions use SpinDoctor's actual calculation and reporting scope.

Term	Definition
Address	A zero-based procedural-family location; distinct from a circuit qubit number or basis index.
Amplitude	A complex coefficient alpha of a basis state. Born probability is |alpha|^2; amplitude itself can be negative or complex.
Attachment	An uploaded input with retained bytes, parsed interpretation, and an ID/hash.
Basis	An orthonormal readout convention. X/Y/Z label Pauli eigenbases; circuit bitstrings follow the declared readout.
Basis index	Decimal integer naming a computational basis bitstring; bit q has weight 2^q.
Bell pair	A maximally entangled two-qubit state; the example prepares (|00>+|11>)/sqrt(2).
Bloch vector / spin	Single-qubit Pauli expectations (x,y,z). Pure single-qubit states have unit length; reduced entangled states can lie inside the sphere.
Born probability	Squared amplitude magnitude; for a subsystem, derived from its reduced density matrix.
Circuit	A named ordered gate program, input basis state, and width. Its definition is separate from execution.
Coherent output	Retained calculated premeasurement state associated with one circuit program hash.
Collapse	State update conditioned on one measurement outcome; tranche collapse requires one shot.
Complex128 / FP64	Two double-precision components per complex amplitude; 16 bytes of dense amplitude storage.
Concurrence	An entanglement measure. The installed query supports a whole pure two-qubit circuit state.
Control / target	A control conditions a gate; the target receives it. Native controlled operands list controls first.
Counter	Recorded position in a sampling/accounting stream. Repeated runs advance counters.
Density matrix	State representation rho; a pure state has rho=|psi><psi|, while a mixed state represents a statistical/local reduction.
Descriptor	Compact procedural formula, dimensions, migration, and shared matrix rather than enumerated states.
Entanglement	Correlations requiring a joint state; the local reduced state can be mixed while the whole state is pure.
Entropy	Quantifies mixedness of a density matrix; for a reduced subsystem of a pure joint state it quantifies entanglement under the declared log base.
Expectation	Mean observable value from a supplied or retained quantum state; not a single sampled bit.
Factor / joint factor	A jointly represented subset of qubits, independent of other factors in the stored state. A connecting gate can merge factors.
Fidelity	State similarity under a declared convention; installed squared fidelity is |<a|b>|^2 for pure states.
Fit / regression	Least-squares relation estimated from selected numeric rows/columns. Residual diagnostics alone are not out-of-sample validation.

Term	Definition
Gate angle	Radians parameterizing an operator; direct terminal gate commands require finite numeric literals.
GHZ	A multi-qubit coherent state (|0...0>+|1...1>)/sqrt(2).
Global phase	A common complex unit phase multiplying a whole state; isolated outcome probabilities do not change.
Hamiltonian	Hermitian operator specifying an energy/evolution model; this parser uses coefficient*Pauli terms.
Histogram	Observed outcome counts from completed shots; compare with theoretical probabilities without expecting identical counts.
Input basis	The computational basis state used before the gate program; default index is zero.
Job	Queued background work with identity, phase, progress, error, and completion metadata.
Logical qubit	A position in the declared coherent register. Its count does not imply a dense arbitrary state fits memory.
Marginal	Probability distribution for a selected qubit subset, summing over unrequested qubits.
Migration t	A procedural-family parameter in [0,1]; not the T gate and not a circuit-width control.
Mixed state	A density matrix with purity below one; can arise as an entangled subsystem reduction or from noise.

REFERENCE 35

34 / Vocabulary dictionary: N-Z

Term	Definition
Normalization	Sum of squared amplitude magnitudes equals one. Most calculator state fields require it explicitly.
Pauli string	Product of labelled Pauli operators such as "Z0 Z1"; labels identify qubits.
Preflight	Validation/prerequisite checks before state-changing execution; availability depends on real backend support.
Prepared-factor-Born-Z	Wide-circuit sampling: one preparation, then independent joint Z draws per coherent factor per shot.
Preset	A saved ready-made program or command sequence. Built-in q.circuit presets differ from qc-001..qc-241 catalogue presets.
Probability	A number in [0,1]; sampled frequency estimates it with statistical variation.
Procedural grid	Formula-defined indexed family of independent qubit states. Address count is not entangled register width.
Program hash	Identity of a circuit definition; retained output must match it to answer state/query requests.
Purity	Tr(rho^2); equals one for a normalized pure state and is lower for mixed states.
QASM	Quantum assembly input language; the installed parser supports a bounded OpenQASM 2.0 subset.
QEC	Quantum error-correction demonstrations under explicit ideal code/error/recovery assumptions.
QFT	Quantum Fourier transform; the built-in program uses its documented positive Fourier convention and final bit-reversal swaps.
Receipt	Archived owner-bound record with stated verified/unverified scope and completion evidence.
Reduced density matrix	State of a selected subsystem obtained by tracing out the other qubits.
Represented operations	Model-described operation equivalents, distinct from actual CPU/GPU work.
Residual	Observed target minus fitted target; helps inspect fit errors on recorded rows.
Seed	Recorded input to a deterministic pseudo-random sampling stream; counter and mode also matter.
Selected circuit	The current named program targeted by ordinary q.circuit and q.query actions.
Sequence	Ordered command array with completion barriers between dependent work.
Shot	One sampling trial under the declared measurement model; requested and completed counts can differ on cancellation/failure.
Sparse / dense	Storage approaches retaining selected entries or every basis amplitude; feasibility depends on actual state structure.
Stabilizer	Pauli operator fixing a state; code calculations depend on commutation, independence, and model assumptions.
Standard error	For a Bernoulli frequency, sqrt(p*(1-p)/shots); a planning/statistical measure, not a result guarantee.

Term	Definition
Tranche	Separate selected coherent register; from I snapshots up to 29 procedural addresses starting at 29*I.
Unitary	Operator preserving norm; gate checks and identity comparisons use explicit numeric tolerances/conventions.
Verification	The specifically declared algebraic or sampled CPU/OpenCL-GPU check; its scope does not automatically cover unrelated calculations.
Virtual epoch	Compact dashboard model step. Its represented-operation arithmetic is separate from measured hardware work.
Workspace	Persistent session folder containing programs, state, input attachments, jobs, counters, and report archives.
ZNE	Zero-noise extrapolation from supplied scale/expectation values; installed linear fit is an educational mitigation calculation.

REFERENCE 36

35 / Source map, version notes, and validation

Source root: `C:/Users/russe/OneDrive/Desktop/SpinDoctor 3.1.2`. This edition uses the updated distribution source/mvp tree and update identity 3.1.2-298-20261010. It preserves distinctions between runtime capability, historical design labels, and older release test reports.

Source within release	Used for
HANDGUIDE.pdf / HANDGUIDE.md	Original tutorial, reports, historical examples, and updated prose.
UPDATE-298.txt / START_HERE.txt	Restart activation, capability identity, width/factor limits, and current scope.
CSV-Input-Guide.txt	CSV table, measurement fitting, equations, and workflow explanations; parser overrides stale width prose.
source/mvp/quantum/QuantumTerminal.cs	Top-level dispatch, procedural model, completion/report behavior.
SpinDoctorSyntax.cs / SpinDoctorCalculators.cs	Validated calculator actions, argument keys, expression grammar, query bounds.
SpinDoctorAdvanced.cs	Noise, QEC, compilation, and algorithm implementation scope.
SpinDoctorCircuits.cs / SpinDoctorTranches.cs / SpinDoctorTrancheTerminal.cs	Coherent register gates, indices, sampling modes, operations, and limits.
SpinDoctorInputs.cs / SpinDoctorSequences.cs / SpinDoctorPdfInputs.cs	Input schemas, sequence normalization, file order, options/defaults, and PDF boundaries.
SpinDoctorCommandReports.cs	History commands and archive behavior.
source/mvp/SPINDOCTOR_PRESETS.json	241 preset records and representative examples.
source/mvp/runtime/virtual-xpu.js	Separate VXPU dashboard parser.
SpinDoctor/PACKAGE_TOOLS.md	Carrier tooling and launcher behavior.

Version notes

Current parser behavior overrides stale prose: direct gate angles use literals, circuit/tranche/gate-CSV width is 298, joint factors stop at 59, and auxiliary analyses retain smaller bounds. Folder purge defaults false; q.sequence run is an editor/API submission. qc-198 synthesis remains unavailable.

Existing saved projection code is refreshed by BOOT. Historical design labels and test reports retain their original build identities; they are not new benchmark evidence.

REFERENCE 37

36 / Validation evidence and practical scope

This guide was checked against the updated local source. Its calculator dictionary examples were also executed in a fresh isolated CPU session, preserving the original distribution and live workspace.

Check	Recorded result
Production source build	13 canonical updated C# source files compiled; product version 3.1.2.
Calculator/advanced dictionary	80/80 exact representative commands passed syntax preflight and native CPU execution; no missing examples or failures.
Bell sampling	128 completed Z shots; only 00 and 11 outcomes; histogram counts total 128.
Bell coherent-state analysis	Retained fidelity 1 with the declared Bell vector; one-qubit purity 1/2.
298-qubit product-H program	16 completed Z shots; 298-bit histogram strings; prepared-factor-Born-Z mode; 298 independent one-qubit factors.
Wide local reduction	Qubit 297 remains a pure local state with the expected query behavior.
Expected parser refusals	Symbolic direct q.gate angle, procedural id alias, CX all, and swapped circuit-run keyword order were refused.
Additional assertions	35 assertions recorded in the validation harness, alongside 80 dictionary example checks.
Narrative and file grammar	Static parser review, including numeric split, names, gate arity, input schemas, options/defaults, and scope.
PDF review	Rendered page images checked for readable tables/code, clipping, overlaps, and page breaks.

What these checks establish

The concrete calculator examples and recorded CPU programs work under the inspected update. Syntax templates still require suitable values, a valid selected state when needed, and the stated prerequisites. Tests did not run OpenCL GPU verification or the maximum folder/memory stress ceilings.

Historical release evidence

The release records include 81 successful original hand-guide examples and the 241-preset audit: 240 completed, with qc-198 preflight-unavailable. Those reports describe their recorded build. They do not assert that every wide entangling program, maximum analysis case, or physical hardware claim was exercised.

Source and output hashes and the validation log are retained in the guide build's working evidence. The guide documents classical simulation, empirical fitting, and scoped verification; virtual or represented counts remain separate from actual executed work.